Final Project Report On

GitHub Repository Analysis

Project Title: GitHub Repository Analysis using Python (EDA & Statistical Analysis)

Tools Used: Python, Pandas, NumPy, Matplotlib, Seaborn, SciPy

Project Type: Exploratory Data Analysis (EDA) + Statistical Analysis + Feature Engineering

Executive Summary

The objective of this project was to convert raw GitHub repository data into a clean, analysis-ready dataset and identify technology trends, repository popularity, and community engagement. The analysis was completed in three sprints: data preparation, exploratory data analysis, and statistical analysis with feature engineering.

The project provides actionable insights into programming language adoption, repository performance, and developer engagement using GitHub metrics such as Stars, Forks, Watchers, and Open Issues.

Data Understanding & Preparation

1. Business Problem

GitHub hosts thousands of repositories across multiple technology domains. Organizations and developers need a structured way to identify popular technologies, highly engaged repositories, and community trends from raw repository data.

2. Business Objective

- Collect GitHub repository data using the GitHub API.
- Clean and validate the raw dataset.
- Analyze repository popularity and engagement.
- Discover technology trends using EDA and statistics.
- Create meaningful business recommendations.

3.Expected Outcome

A clean, structured dataset that helps identify:

- Popular programming languages
- High-performing repository categories
- Community engagement trends
- Repository maintenance patterns

3. Dataset Understanding

Attribute	Description
Category	Technology domain
Topic	Repository topic
Repository	Repository name
Owner	Repository owner
Language	Programming language
Stars	Popularity metric
Forks	Community contribution
Watchers	User engagement
Open Issues	Maintenance activity
Created At	Creation date
Updated At	Last update

4. Data Cleaning Performed

Issue	Action Taken
Missing Language values	Filled with "Unknown"
Duplicate records	Removed
Date columns	Converted to datetime
Invalid numeric values	Validated
Inconsistent text	Trimmed categorical values

5. Outlier Analysis

Method Used: IQR + Box Plot

Decision: Outliers were retained because repositories with extremely high stars represent genuine business value rather than data errors.

6. Data Validation

- No duplicate records
- Appropriate data types
- Clean categorical values
- Analysis-ready dataset

Exploratory Data Analysis

1. Descriptive Statistics

Analyzed numerical variables:

- Stars
- Forks
- Watchers
- Open Issues

Calculated:

- Mean
- Median
- Mode
- Standard Deviation
- Variance
- Minimum & Maximum

Observation

Repository popularity is highly variable, indicating that only a small number of projects receive massive community attention.

2. Univariate Analysis

Numerical Variables

- Distribution of Stars
- Distribution of Forks
- Outlier identification

Categorical Variables

- Programming Language frequency
- Category distribution
- Topic frequency

Key Findings

- Python appears frequently in Data Science.
- JavaScript dominates Web Development.
- Stars show positive skewness.
-

3. Bivariate Analysis

Variables	Purpose
Language vs Stars	Popularity comparison
Category vs Stars	Domain performance
Stars vs Forks	Community relationship
Stars vs Watchers	Engagement analysis

Major Finding : Repositories with higher Stars generally receive more Forks and Watchers.

4. Multivariate Analysis

Simultaneously analyzed:

- Stars
- Forks
- Watchers
- Open Issues

Technique: Pair Plot & Heatmap

Finding : Community engagement metrics move together, indicating strong relationships between popularity and contribution.

5. Group-Based Analysis

Grouped by:

- Category
- Language

Example Business Questions

- Which language has highest average stars?
- Which category receives maximum engagement?
- Which domains have active repositories?

6. Pivot Table Analysis

Created pivot tables comparing:

- Category vs Language
- Language vs Average Stars
- Language vs Average Forks

Business Value : Pivot tables summarized repository performance from multiple perspectives.

7. Crosstab Analysis

Analyzed relationships between:

- Category
- Programming Language

This identified which technologies dominate specific domains.

8. Correlation Analysis

Relationship	Result
Stars ↔ Forks	Strong Positive
Stars ↔ Watchers	Strong Positive
Stars ↔ Open Issues	Weak
Forks ↔ Watchers	Positive

Interpretation

Higher popularity leads to greater community participation.

9. Visualizations Used

Chart	Business Question
Histogram	How are stars distributed?
Count Plot	Which languages are most common?
Box Plot	Which categories contain outliers?
Violin Plot	Distribution of forks
Scatter Plot	Do forks increase with stars?
Bar Chart	Average stars by category
Pair Plot	Numerical relationships
Heatmap	Correlation strength

10. Visual Storytelling Example

Chart: Scatter Plot (Stars vs Forks)

Observation: Repositories with higher stars tend to have higher forks.

Interpretation:

Popular projects encourage community contributions.

Business Insight: Organizations should actively maintain high-quality repositories to increase contributor engagement.

Business Insights

- Python dominates Data Science repositories.
- JavaScript leads Web Development.
- Stars are a strong indicator of engagement.
- Highly maintained repositories attract more watchers.

Statistical Analysis & Feature Engineering

1. Statistical Analysis

Performed:

- Measures of Central Tendency
- Measures of Dispersion
- Distribution Analysis
- Correlation
- Covariance

Interpretation

The dataset exhibits right-skewed popularity, with strong positive relationships among engagement metrics.

2. Hypothesis Testing

Test 1 – Independent T-Test

Question: Do Python and Java repositories have different average stars?

- H_0: Mean stars are equal.
- H_1: Mean stars are different.

Decision Rule

- $p < 0.05 \rightarrow$ Reject H_0
- $p \geq 0.05 \rightarrow$ Fail to Reject H_0

Business Interpretation: Programming language may influence repository popularity.

Test 2 – Chi-Square Test

Question: Is Category associated with Programming Language?

- H_0: Independent
- H_1: Associated

Business interpretation depends on the p-value.

Test 3 – ANOVA

Question: Do repository categories have different average stars?

ANOVA compares multiple categories simultaneously to determine significant differences.

3. Confidence Interval

Constructed a 95% Confidence Interval for Stars.

Interpretation : We are 95% confident that the true average repository stars lie within the calculated interval.

4. Feature Engineering

Created new business-friendly features.

Feature	Purpose
Popularity Category	Segment repositories
Repository Age	Measure maturity
Active Repo Flag	Identify maintained projects
Engagement Score	Combined engagement metric

Engagement Score

$$Stars + (2 \times Forks) + Watchers$$

This provides a more comprehensive popularity measure than Stars alone.

5. Feature Evaluation

Compared original and engineered features.

Evidence

- Better segmentation of repositories
- Easier comparison across technologies
- Improved business interpretation

6. Final Business Recommendations

Recommendation 1

Focus on Python projects for Data Science initiatives due to strong community engagement.

Recommendation 2

Maintain repositories with frequent updates to increase Watchers and contributor trust.

Recommendation 3

Improve documentation to encourage Forks and open-source contributions.

Recommendation 4

Monitor Open Issues regularly to improve repository quality and developer satisfaction.

Recommendation 5

Use Engagement Score instead of Stars alone when evaluating project success.

Overall Business Insights

Key findings

Popularity

Stars remain the strongest indicator of repository visibility.

Community

Forks increase alongside Stars, showing contributor engagement.

Technology

Python and JavaScript dominate their respective domains.

Maintenance

Recently updated repositories attract stronger community attention.

Conclusion

This analysis successfully transformed raw GitHub repository data into a clean, structured, and analysis-ready dataset through systematic data preparation, exploratory data analysis, statistical analysis, and feature engineering.

The findings revealed that Python and JavaScript are the most influential programming languages in their respective domains, while Stars, Forks, and Watchers are strongly associated with community engagement. Statistical analysis and hypothesis testing provided evidence-based insights into repository performance, and the engineered features enabled better segmentation and evaluation of repositories.

Overall, the analysis demonstrates how data can be used to uncover technology trends, measure developer engagement, and support informed decision-making for open-source communities and organizations.

THANK YOU